

Inter Process Communication

- The communication of more than one process with another process for making a program is known as the inter process communication.
- IPC is divided into pipes, FIFOs, message queues, semaphores, and shared memory.

Pipes

- Pipes are the oldest form of UNIX System IPC and are provided by all UNIX systems. Pipes have two limitations.
 - 1.They have been half duplex.
 - 2.Pipes can be used only between processes that have a common ancestor.
- A pipe is a one-way mechanism that allows two related processes to send data from one of them to the other one.

Pipes

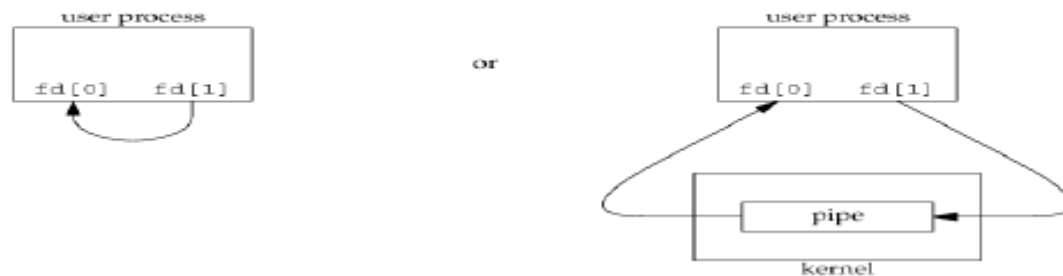
- A pipe is created by calling the pipe function.

```
#include <unistd.h>
int pipe(int filedes[2]);
```

Returns: 0 if OK, -1 on error.

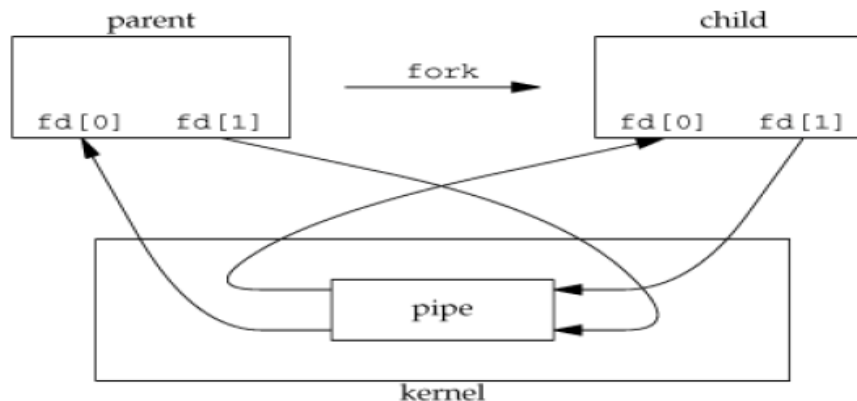
- Two file descriptors are returned through the filedes argument: `filedes[0]` is open for reading, and `filedes[1]` is open for writing. The output of `filedes[0]` is the input for `filedes[1]`.
- Two ways to picture a half-duplex pipe are shown in Figure 15.2. The left half of the figure shows the two ends of the pipe connected in a single process. The right half of the figure emphasizes that the data in the pipe flows through the kernel

Figure 15.2. Two ways to view a half-duplex pipe



Pipes

- Normally, the process that calls pipe then calls fork, creating an IPC channel from the parent to the child or vice versa.
- What happens after the fork depends on which direction of data flow we want. For a pipe from the parent to the child, the parent closes the read end of the pipe (fd[0]), and the child closes the write end (fd[1]).
- For a pipe from the child to the parent, the parent closes fd[1], and the child closes fd[0]. When one end of a pipe is closed, the following two rules apply.
 - If we read from a pipe whose write end has been closed, read returns 0 to indicate an end of file after all the data has been read.
 - If we write to a pipe whose read end has been closed, the signal SIGPIPE is generated. If we either ignore the signal or catch it and return from the signal handler, write returns 1 with errno set to EPIPE.



Pipes

popen and pclose :

A common operation is to create a pipe to another process, to either read its output or send it input, the standard I/O library has historically provided the popen and pclose functions.

These two functions handle all the dirty work that we've been doing ourselves: creating a pipe, forking a child, closing the unused ends of the pipe, executing a shell to run the command, and waiting for the command to terminate.

```
#include <stdio.h>
```

```
FILE *popen (const char *cmdstring, const char *type);
```

Returns: file pointer if OK, NULL on error.

```
int pclose(FILE *fp);
```

Returns: termination status of cmdstring, or -1 on error

Pipes

- The function `popen` does a `fork` and `exec` to execute the `cmdstring`, and returns a standard I/O file pointer. If type is "r", the file pointer is connected to the standard output of `cmdstring`.

Figure 15.9. Result of `fp = popen(cmdstring, "r")`



If type is "w", the file pointer is connected to the standard input of `cmdstring`.

Figure 15.10. Result of `fp = popen(cmdstring, "w")`



- The `pclose` function closes the standard I/O stream, waits for the command to terminate, and returns the termination status of the shell.

FIFOs

- FIFOs are sometimes called named pipes. Pipes can be used only between related processes when a common ancestor has created the pipe. With FIFOs, unrelated processes can exchange data.

Creating a FIFO is similar to creating a file.

```
#include <sys/ stat.h>  
int mkfifo( const char *pathname, mode_t mode);
```

Returns: 0 if OK,-1 on error.

- Once we have used mkfifo to create a FIFO, we open it using open. When we open a FIFO, the nonblocking flag (O_NONBLOCK) affects what happens.
 - In the normal case (O_NONBLOCK not specified), an open for read-only blocks until some other process opens the FIFO for writing. Similarly, an open for write-only blocks until some other process opens the FIFO for reading.
 - If O_NONBLOCK is specified, an open for read-only returns immediately. But an open for write-only returns 1 with errno set to ENXIO if no process has the FIFO open for reading.

FIFOs

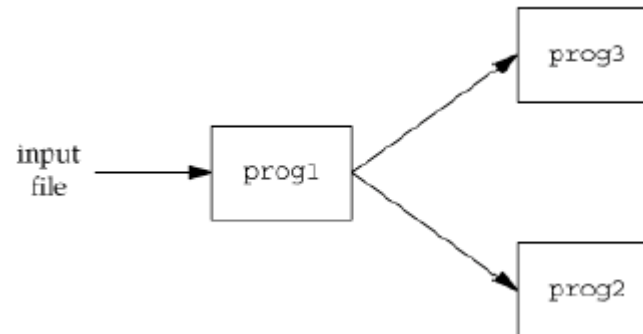
Uses for FIFOs:

- FIFOs are used by shell commands to pass data from one shell pipeline to another without creating intermediate temporary files.
- FIFOs are used in client-server applications to pass data between the clients and the servers.

FIFOs

Example Using FIFOs to Duplicate Output Streams

- FIFOs can be used to duplicate an output stream in a series of shell commands. This prevents writing the data to an intermediate disk file. Consider a procedure that needs to process a filtered input stream twice. Figure 15.20 shows this arrangement.



Procedure that processes a filtered input stream twice

- With a FIFO and the UNIX program `tee(1)`, we can accomplish this procedure without using a temporary file. (The `tee` program copies its standard input to both its standard output and to the file named on its command line.)

```
mkfifo fifo1
```

```
prog3 < fifo1 &
```

```
prog1 < infile | tee fifo1 | prog2
```

FIFOs

- We create the FIFO and then start prog3 in the background, reading from the FIFO. We then start prog1 and use tee to send its input to both the FIFO and prog2. Figure 15.21 shows the process arrangement.

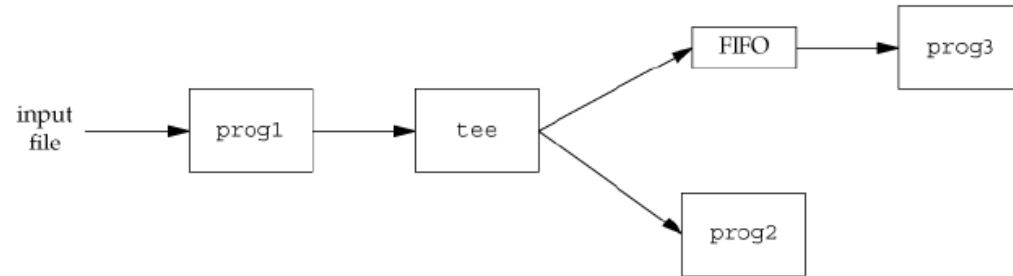


FIGURE 15.21 : Using a FIFO and `tee` to send a stream to two different processes

Introduction to three types of IPC

There are three types of IPC that are called Unix system V IPC. They are:

- 1) Message queues
- 2) Semaphores
- 3) Shared Memory

Message Queues:

- A message queue is a linked list of messages stored within the kernel and identified by a message queue identifier. We'll call the message queue just a queue and its identifier a queue ID.
- Message queues allow one or more processes to write message to be read by other processes.
- A new queue is created or an existing queue opened by *msgget*. New messages are added to the end of a queue by *msgsnd*. Every message has a positive long integer type field, a non-negative length, and the actual data bytes all of which are specified to *msgsnd* when the message is added to a queue.

Message Queues:

- It also maintains, the following information for each message queue in the system,
 - i) Owner of the queue and access permissions for this queue.
 - ii) Pointers to the first and last messages on a queue.
 - iii) Total no. of messages on a queue.
 - iv) Total no. of data bytes on a queue.
 - v) The maximum no. of bytes allowed on a queue.
 - vi) The process IDs of the last processes that sent messages to and received messages from the queue.
 - vii) the time stamp of the last message sent (msgsnd), received (msgrev) and controlled (msgctl).
- There are several system limits that affect message queues. These limits are defined in the “sys/msg.h” header file.

System-limit	Meaning	Value
MSGMNI	The maximum no. of message queues allowed in system	50
MSGTQL	The maximum no. of messages allowed in a system	40
MSGMNB	The maximum no. of data bytes allowed for all messages in a queue.	4096
MSGMAX	The maximum no. of data bytes allowed for a message.	2048

Semaphores :

- Semaphores allow multiple processes to synchronize their execution.

A semaphore is a counter used to provide access to a shared data object for multiple processes

To obtain a shared resource, a process needs to do the following:

1. Test the semaphore that controls the resource.
2. If the value of the semaphore is positive, the process can use the resource. In this case, the process decrements the semaphore value by 1, indicating that it has used one unit of the resource.
3. If the value of the semaphore is 0, the process goes to sleep until the semaphore value is greater than 0. When the process wakes up, it returns to step 1.

Semaphores :

- Semaphores are allocated in one or more sets.
- A process can use multiple semaphore sets. The kernel keeps track of all the semaphore sets created in a system.
- For each semaphore set, the kernel maintains the following information,
 - i) Owner of the set and access permissions for this set.
 - ii) A pointer to the first semaphore in a set.
 - iii) The number of semaphores in a set.
 - iv) The time at which last changes were made to one or more semaphore values.
 - v) The time at which last changes were made to the control data of the set.
 - vi) the current value of the semaphore, which is always greater than or equal to zero.
 - vii) the process ID of the last process that did an operation on the semaphore.
 - viii) the number of processes waiting for the semaphore value to increase.
 - ix) The number of processes waiting for the semaphore value to become zero.

Semaphores :

- A new semaphore set is created using the `semget()` system call.
- The `semctl()` and `semop()` functions are used to control and manipulate operations on semaphore.
- There are several system limits that affect semaphore as shown in table. These limits are defined in the “`sys/sem.h`” header file.

System-limit	Meaning	Value
SEMMNI	The maximum no. of semaphore sets allowed in a system	10
SEMMNS	The maximum no. of semaphores in all sets allowed in a system	60
SEMMSL	The maximum no. of semaphores allowed in a set.	25
SEMVMX	The maximum positive value of any semaphore	32,767
SEMAEM	The max. adjust on exist value of any semaphore	16,384

Shared Memory

- Shared memory allows two or more processes to share a given region of memory. This is the fastest form of IPC, because the data does not need to be copied between the client and the server.
- The kernel keeps track of all the shared memory regions created in a system. For each shared memory region, the kernel maintains the following information:
 - i) Owner and the access permission for this regions.
 - ii) A pointer in the kernel.
 - iii) The size of shared memory region in bytes.
 - iv) The number of times the region is locked.
 - v) The process ID of the last processes that created and manipulated the regions.
 - vi) The number of processes attached to the region and its related information.
 - vii) The time at which the last process attached to and detached from the region.
 - viii) The time at which the last process changed control data of the region.

Shared Memory

- The shared memory regions created using shmget system call.
- The shmat and shmdt attaches and detaches region from the address space of calling process respectively.
- There are several system limits that affects shared memory. These limits are defined in the “sys/shm.h” header file.

System-limit	Meaning	Value
SHMMNI	The maximum no. of shared memory regions allowed in a system	100
SHMSEG	The maximum no. of shared memory regions allowed for any process	6
SHMMIN	The maximum no. of bytes allowed in a shared memory region.	1
SHMMAX	The maximum no. of bytes allowed in a shared memory region	131,072

Message Queues

- A message queue is a linked list of messages stored within the kernel and identified by a message queue identifier. We'll call the message queue just a queue and its identifier a queue ID.
- A new queue is created or an existing queue opened by `msgget`. New messages are added to the end of a queue by `msgsnd`.
- Every message has a positive long integer type field, a non-negative length, and the actual data bytes (corresponding to the length), all of which are specified to `msgsnd` when the message is added to a queue. Messages are fetched from a queue by `msgrcv`.
- We don't have to fetch the messages in a first-in, first-out order. Instead, we can fetch messages based on their type field. Each queue has the following `msqid_ds` structure associated with it:

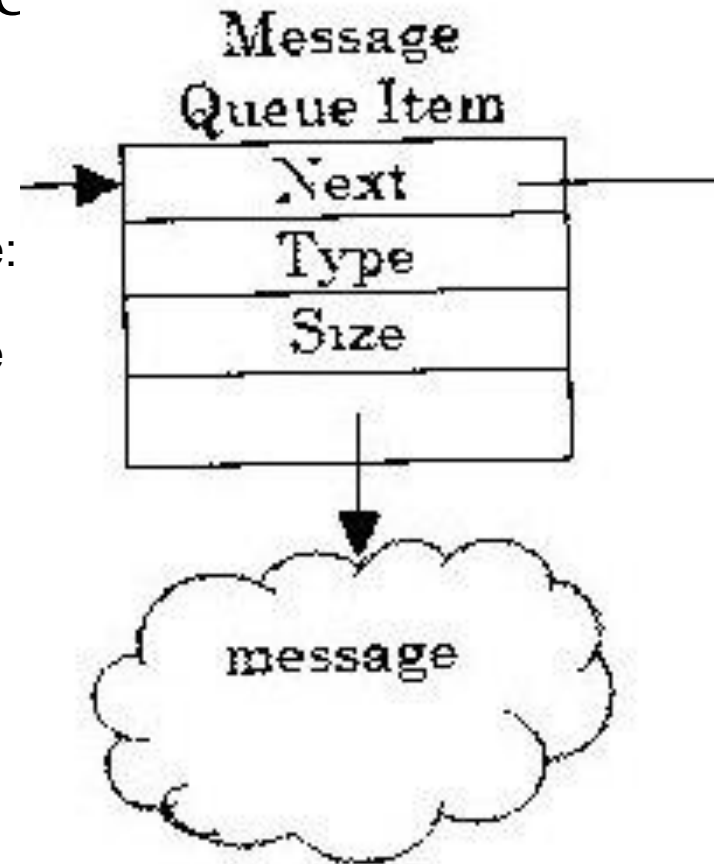
Message Queues

- One process establishes a message queue that others may access. Often a server will establish a message queue that multiple clients can access
- Features of Message Queues
 - A process generating a message may specify its type when it places the message in a message queue.
 - Another process accessing the message queue can use the message type to selectively read only messages of specific type(s) in a first-in-first-out manner.
 - Message queues provide a user with a means of multiplexing data from one or more producer(s) to one or more consumer(s).

Message Queues

Attributes of Message

- A conceptual view of a message queue, from *Interprocess Communications in Unix*:
 - The attributes of each element on the queue:
 - long integer **type**;
 - **size** of the data portion of the Message (can be zero);
- * A message queue element then has one more field:
 - **data** (if the length is greater than zero)
- * Message Structure



Message Queues

Message Queue Structure

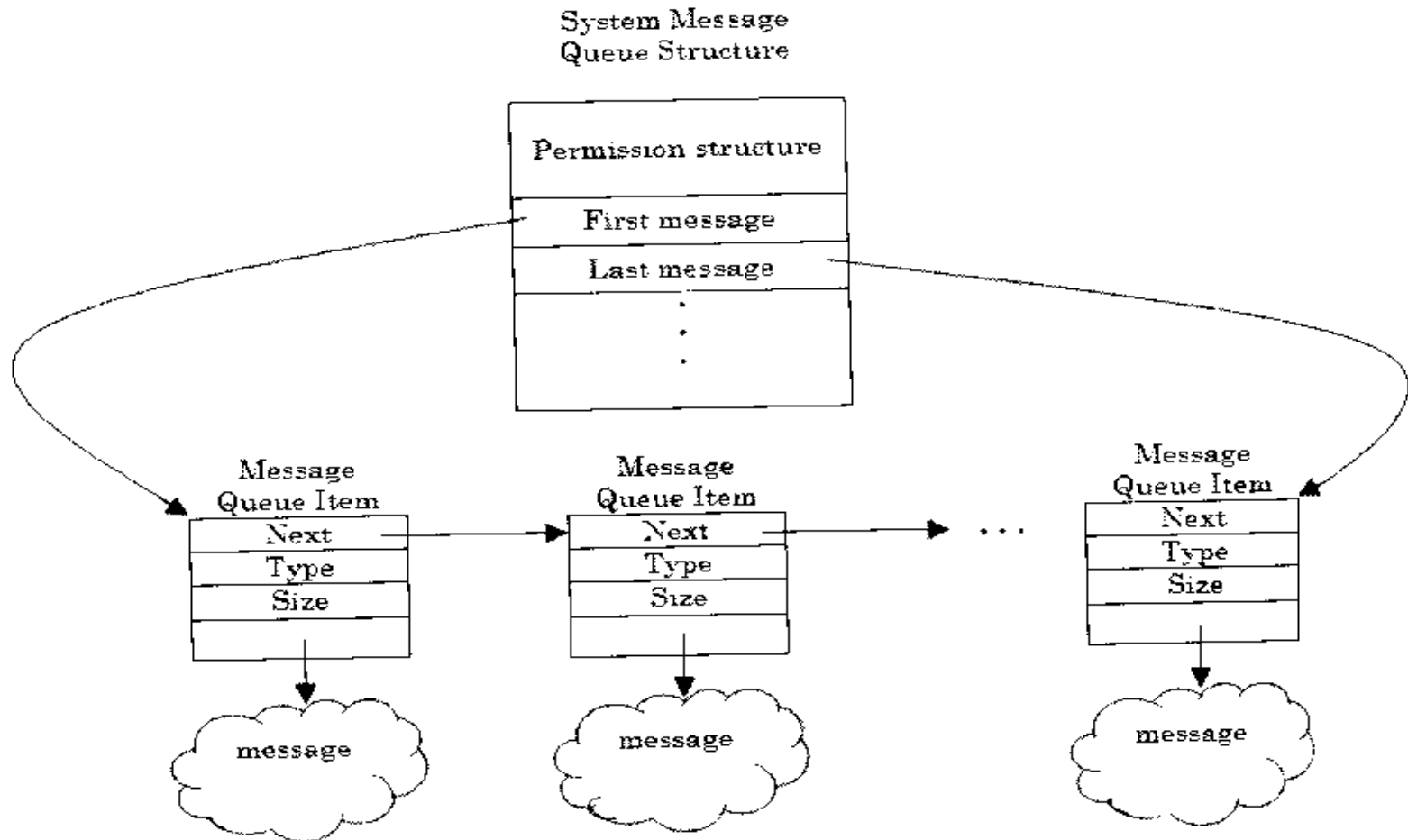


Figure 6.5 A message queue with N items.

Message Queues

msqid_ds Structure

```
struct msqid_ds {  
    struct ipc_perm  msg_perm; /*operation permission struct */  
    struct msg  *msg_first; /* pointer to first message on q*/  
    struct msg  *msg_last; /* point to last message on q */  
    ulong        msg_cbytes; /* current # bytes on q */  
    ulong        msg_qnum; /* # of message on q */  
    ulong        msg_qbytes; /* max # of bytes on q */  
    pid_t        msg_lspid; /* pid of last msgsnd */  
    pid_t        msg_lrpid; /* pid of last msgrcv */  
    time_t       msg_stime; /* last msgsnd time */  
    .....  
}; /* total 17 members */
```

Message Queues

ipc_perm Structure

```
•struct ipc_perm {  
    uid_t      uid;      /*owner's user ID */  
    gid_t      gid;      /* owner's group ID */  
    uid_t      cuid;     /* creator's user ID */  
    gid_t      cgid;     /* creator's group ID */  
    mode_t mode; /* access modes */  
    ulong      seg;      /* slot usage sequence number */  
    key_t      key;      /* key */  
    long       pad[4]; /* reserve area */  
};  
•Struct msqid_ds {  
    struct ipc_perm      msg_perm; .....};
```


Message Queues

msg structure

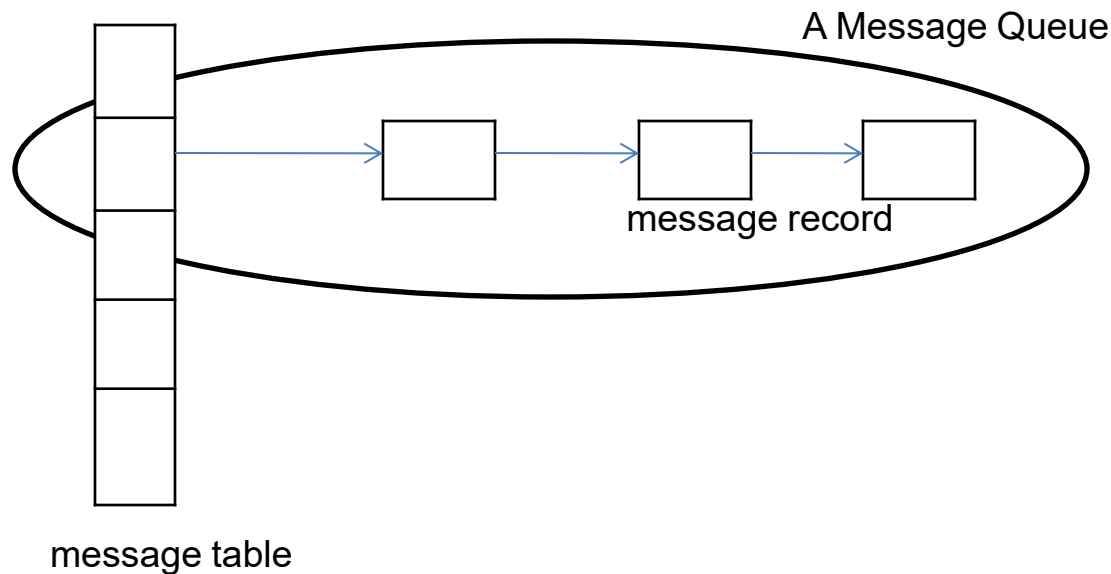
```
struct msg {  
    struct msg  *msg_next; /* pointer to next message on q */  
    long        msg_type; /* message type */  
    ushort     msg_ts;    /* message text size */  
    short       msg_spot; /* message text map address */  
};
```

UNIX Kernel Support for Messages

- Specifically, there is a message queue table in a kernel address space that keeps track of all message queues created in a system.
- Each entry of the message tables stores the following data for one message queue:
 - A name, which is an integer ID key assigned by the process that created the queue. Other processes may specify this key to “open” the queue and gets a descriptor for future access of the queue
 - The creator user ID and group ID. A process whose effective user ID matches a message queue creator user ID may delete the queue and also change the queue control data.
 - The assigned owner user ID and group ID. These are normally the same as those of the creator, but a creator can set these values to reassign the queue owner and group membership.
 - Read-write access permission of the queue for owner, group, or others.
 - The time and process ID of the last process that sent a message to the queue.
 - The time and process ID of the last process that read a message from the queue.
 - The pointer to a linked list of message records in the queue.

UNIX Kernel Support for Messages

- When a process sends a message to a queue, the kernel creates a new message record and puts it at the end of the message record linked list for the specified queue.



Kernel Data structure for message queues

UNIX Kernel Support for Messages

- The message record stores the message type, the number of bytes of the message data, and the pointer to another kernel data region, where the actual message data is stored.
- The kernel copies the message data from the sender process's virtual address into the kernel data region, so that the sender process is free to terminate, and the message can still be read by another process in the future.
- When a process retrieves a message from a queue, the kernel copies the message data from a message record to the process's virtual address and then discards the message record.
- The process can retrieve a message in a queue in the following manners:
 - Retrieve the oldest message in the queue, regardless of its message type.
 - Retrieve a message whose message ID matches the one specified by the process. If there are multiple messages with the given message type existing in the queue, retrieve the oldest one among them.
 - Retrieve a message whose message type is the lowest among those that are less than or equal to the one specified by the process. If there are multiple messages that satisfy the same criteria, retrieve the oldest one among them.

UNIX System V APIs for Messages

- There are four different Unix system V APIs for messages,

1. `msgget()`
2. `msgsend()`
3. `msgrcv()`
4. `msgctl()`

`msgget()`:

The first function normally called is `msgget` to either open an existing queue or create a new queue.

The function prototype of the *msgget* API is:

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/message.h>
```

```
int msgget ( key_t key, int flag);
```

This function “opens” a message queue whose key ID matches the key actual value and returns a positive integer descriptor. This can be used in other message APIs to send and receive messages and to query and/or set control data for the queue.

UNIX System V APIs for Messages

- For example, the following system call creates a message queue with the key ID of 15 and access permission of 0644 (that is, read-write for owner and read-only for group members and others), if such a queue does not preexist. The call also returns an integer descriptor for future queue references:

```
int msgfdesc = msgget ( 15, IPC_CREAT|0644);
```

- If a process wishes to guarantee the creation of a new message queue, it can specify the IPC_EXCL flag with the IPC_CREAT flag, and API will succeed only if it creates a new queue with the given *key*.
- The API return -1 if it fails. Some possible causes of failure are:

<i>Errno</i> Value	Cause of error
ENOSPC	The system-imposed limit MSGMNI has been reached
ENOENT	The <i>flag</i> value does not contain the IPC_CREAT flag, and no queue exists with the specified <i>key</i> .
EEXIST	The IPC_EXCL and IPC_CREAT flags are set in the <i>flag</i> value, and a message queue with the specified <i>key</i> already exist
EACCESS	The queue with the specified <i>key</i> exists, but the calling process has no access permission to the queue

UNIX System V APIs for Messages

- `msgsnd`

This system call is used for sending message in an Unix environment.

The function prototype of the *msgsnd* API is:

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg.h>
```

```
int msgsnd ( int msgfd, const void* msgPtr, int len,int flag);
```

This API sends a message (pointed to by *msgPtr*) to a message queue designated by the *msgfd* descriptor.

The *msgfd* value is obtained from a *msgget* function call.

The actual value of the *msgPtr* argument is the pointer of an object that contains the actual message text and a message type to be sent. The following data type can be used to define an object for such purpose:

```
struct msgbuf
{
    long mtype; //message type
    char mtext[MSGMAX]; //buffer to hold the message text
};
```

UNIX System V APIs for Messages

- The *len* value is the size, in bytes, of the *mtext* field of the object pointed to by the *msgPtr* argument.
- The *flag* value may be 0, which means the process can be blocked, if needed, until the function call completes successfully. If it is the IPC_NOWAIT flag, the function aborts if the process is to be blocked.
- The return value of the API is 0 if it succeeds or -1 if it fails.

msgrcv

The function prototype of the *msgrcv* API is :

```
#include<sys/types.h>
#include<sys/ipc.h>
#include<sys/msg.h>
```

```
int msgrcv ( int msgfd, const void* msgPtr, int len,int mtype, int flag);
```

This API receives a message of type *mtype* from a message queue designated by the *msgfd*. The message received is stored in the object pointed to by the *msgPtr* argument. The *len* argument specifies the maximum number of message text bytes that can be received by this call.

The *msgfd* value is obtained from a *msgget* function call

UNIX System V APIs for Messages

- The *mtype* value is the message type of the message to be received. The possible values and meaning of this argument are:

<i>mtype value</i>	<i>Meaning</i>
0	Receive the oldest message of any type in the queue
+ve integer	Receive the oldest message of the specified message type
-ve integer	Receive a message whose message type is less than or equal to the absolute value of the mtype. If there is more than one message in the queue meeting this criteria, receive the one that is the oldest and has the smallest message type value

UNIX System V APIs for Messages

msgctl System Call - Message Queue Control

Function

Ownership and access permissions, established when the message queue was created, can be examined and modified using the *msgctl* system call.

Include

< sys/types.h> <sys/ipc.h> < sys/msg.h>

Command:

int msgctl (int msqid, int cmd, struct msqid_ds * buf);

Return

Success: 0; Failure: -1; Sets errno: Yes

UNIX System V APIs for Messages

Arguments of *msgctl* System Call

- *int msqid*

- ✓ Message queue identifier (generated by *msgget* call)

- *int cmd*

- ✓ Indicates one of the following actions:

- IPC_STAT: return the current value for each member of the *msgid_ds* data structure (contains the permission structure).

- IPC_SET: the user can modify a limited number of

- msgid_ds* structure member value, such as *msg_perm.uid*; *msg_perm.gid*; *msg_perm.mode*; and *msg_qbytes*.

- IPC_RMID: removes all associated message queue structure.

*struct msgid_ds *buf* : pointer to a *msgid_ds* struct

UNIX System V APIs for Messages

Remove a Message Queue in a Program

Command:

*msgctl(msqid, IPC_RMID, (struct msqid_ds *) 0);*

To remove the message queue with key *msqid*.

You must be the owner

Client/Server Example

- Client / Server example using Message queues is shown below in text file
- Here we are considering a server process and a single client process
- with either pipes or FIFO's, two IPC channels are required to exchange data in both directions.
- Since these types of IPC's are unidirectional with a message queue, a single queue can be used, having then type of each message signify if the message is from the client to the server to the client .
- The client process obtains input from the keyboard and sends it via message queue to the server.
- server reads the message from the message queue.